
DSC 40B - Homework 04

Due: Wednesday, April 29

Write your solutions to the following problems by handwriting them on another piece of paper. Unless otherwise noted by the problem's instructions, show your work or provide some justification for your answer. Homeworks are due via Gradescope at 11:59 p.m.

Problem 1.

What is the *best* case time complexity of `quickselect`? Describe why and provide the input that yields this best case. For simplicity, you may assume that `Quickselect` always chooses the last element of the input array as the pivot (that is, the pivot is *not* chosen randomly).

Problem 2.

Refer to the implementation of `quickselect` in lecture.

Suppose we use `quickselect` to select the minimum element of the array $A = [3, 2, 9, 0, 7, 5, 4, 8, 6, 1]$. Describe a sequence of partitions (which pivot index is chosen at each step) that results in a worst-case performance of `quickselect`.

Problem 3.

An understanding of data structures and the time complexity of the various operations involving them is crucial when designing algorithms. Using the wrong data structure, or using the right data structure in the wrong way, can lead to an algorithm which is still correct, but significantly slower. We will see one example in this problem.

Another way of implementing the `merge` function used in `mergesort` is shown below:

```
def merge(left, right, arr):
    left.append(float('inf'))
    right.append(float('inf'))
    for ix in range(len(arr)):
        if left[0] < right[0]:
            arr[ix] = left.pop(0)
        else:
            arr[ix] = right.pop(0)
```

This code does essentially the same thing as the `merge` we saw in class: it merges two sorted lists `left` and `right` into a list `arr` such that the result is in sorted order. However, instead of using index variables to keep track of the front of `left` and `right`, this code uses `list.pop(0)` to remove the first element from the appropriate list.

- Popping an element from the end of a Python list is quick and efficient, but popping the first element from a list requires that the remaining elements be shifted over to the left by one place. Shifting requires copying the elements, one by one, to their new location. What is the time complexity of `arr.pop(0)`, assuming that `arr` is a list of size n ?
- What is the time complexity of the new version of `merge`, assuming that (when they are passed to the function) `left` and `right` are lists of size $n/2$, and `arr` is a list of size n ? Show your work by writing a formula for runtime of the function and its asymptotic runtime.

- c) What is the time complexity of a `mergesort` which uses the above version of `merge` instead of the linear time version used in lecture? Show your work by showing the new recurrence relation and solving it.

Programming Problem 1.

Suppose you are trying to remove outliers from a data set consisting of points in $\mathbb{R}^d$. One of the simplest approaches is to remove points that are in “sparse” regions – that is, points that don’t have many other points close by. To do this, we might calculate the distance from a point to its k th closest neighbor. If this distance is above some threshold, we consider the point an outlier.

More generally, the task of finding the distance from a query point to its k th closest “neighbor” is a common one in data science and machine learning. Here, we’ll consider the 1-dimensional version of the problem of finding k th neighbor distance. In a file named `knn_distance.py`, write a function named `knn_distance(arr, q, k)` that returns a pair of two things:

- the distance between `q` and the k th closest point to `q` in `arr`;
- the k th closest point to `q` in `arr` itself

The query point `q` does not need to be in `arr`. For simplicity, `arr` will be a Python list of numbers, and `q` will be a number. `k` should start counting at one, so that `knn_distance(arr, q, 1)` returns the distance between `q` and the point in `arr` closest to `q`. Your approach should have an expected time of $\Theta(n)$, where n is the size of the input list. Your function may modify `arr`. In cases of a tie, the point you return is arbitrary (though the distance is not). Your code can assume that k will be $\leq \text{len}(\text{arr})$.

Example:

```
>>> knn_distance([3, 10, 52, 15], 19, 1)
(4, 15)
>>> knn_distance([3, 10, 52, 15,], 19, 2)
(9, 10)
>>> knn_distance([3, 10, 52, 15], 19, 3)
(16, 3)
```

As this is a programming problem, submit your code to the Gradescope autograder.